

CSCE 713 SOFTWARE SECURITY · MODULE 09 · LECTURE NOTES

Sanitisers and Runtime Detection

Making silent corruption loud.

MODULE OUTCOMES

- Explain what each sanitiser detects and how.
- Explain the cost and the incompatibilities.
- Explain why sanitisers and fuzzing are a pair.
- Explain what sanitisers cannot detect.
- Deploy them in a real build and test pipeline.

1 AddressSanitizer

ADDRESSSANITIZER MAKES OUT-OF-BOUNDS AND USE-AFTER-FREE LOUD INSTEAD OF SILENT

It places poisoned 'redzones' around every allocation and checks every memory access against a shadow map that records which bytes are valid — so an access into a redzone is reported immediately, with both the allocation site and the access site located precisely, which is what makes the reports actionable.

And it quarantines freed memory rather than returning it to the allocator for immediate reuse, so a use-after-free hits poisoned memory and is reported, instead of silently reading or writing somebody else's live object (Module 02 §2).

Which is precisely why its limits are what they are: an overflow that lands *inside another valid object* is not detected, because that memory is legitimately mapped and unpoisoned — so a large jump past the redzone, or an index far out of range, can be missed entirely while a one-byte overflow is caught.

But it detects the overwhelming majority of real defects, because real defects are mostly small overshoots — and it requires no source changes at all, only a compiler flag, which makes enabling it the single highest-value action available to an existing C or C++ project and the thing to do before anything else in this course.

Sanitiser	What it detects	What it costs
AddressSanitizer (ASan)	Out-of-bounds read and write, use-after-free, double free, use-after-return, and leaks.	Roughly 2× time and 3× memory — acceptable for testing, generally not for production.

Sanitiser	What it detects	What it costs
UndefinedBehaviorSanitizer (UBSan)	Signed overflow, invalid shifts, misalignment, invalid casts, null dereference.	Low — frequently under 20%, and see the note.
ThreadSanitizer (TSan)	Data races (Module 05 §1).	Roughly 5 to 15× time and substantial memory — run it over the concurrency tests specifically.
MemorySanitizer (MSan)	Reads of uninitialised memory.	Roughly 3×, and it requires every dependency to be instrumented — which is the practical obstacle.
Hardware-assisted ASan (HWASan)	Much the same as ASan, using pointer tagging.	Low enough for production use on ARM, which is §4's opportunity.

ASan and TSan are mutually incompatible and must be separate builds — they both instrument memory access and their runtimes conflict — which means a complete pipeline has at least two instrumented configurations. And UBSan's cost is low enough that parts of it ship in production in some projects, which makes it a hardening measure as well as a debugging one (§2).

2 UndefinedBehaviorSanitizer

- **Signed integer overflow** — which is Module 03 §1's canonical defect and is otherwise entirely silent, since the wrapped value is a perfectly ordinary number as far as the program is concerned.
- **Shifts by more than the operand width**, which is undefined and produces platform-dependent nonsense — and which appears regularly in bit-manipulation code written against one compiler.
- **Misaligned and null pointer dereferences**, and **invalid casts between pointer types** — which is Module 03 §4's **type confusion**, caught at the moment the invalid interpretation occurs.
- **Array bounds where the size is statically known**, and **invalid enum or boolean values** — a boolean holding something other than 0 or 1, which arises from uninitialised memory or a bad cast and then breaks every branch on it.
- **And it can be configured to trap rather than to report**, which turns undefined behaviour into a **deterministic abort** — a hardening measure rather than a debugging one (Module 12 §2). **The trap mode is the underused feature: -fsanitize-trap converts a class of exploitable undefined behaviour into a clean crash, at almost no runtime cost**, and is deployed in production by several major projects.

3 The pairing

FUZZING GENERATES THE INPUTS AND SANITISERS SUPPLY THE ORACLE

Fuzzing without a sanitiser only detects defects that actually crash — and a **one-byte out-of-bounds read does not crash**, it returns whatever was there, **so the input that triggered it is discarded as uninteresting** and the defect is never found however long the campaign runs.

A sanitiser without fuzzing only checks the inputs your tests happen to supply — which are the valid, expected, well-formed ones, **and are precisely not where the defects are** (Module 01 §2's input framing).

Together, the fuzzer explores the input space while the sanitiser reports every violation on every path it reaches — which is why **this specific pairing found most of the memory safety defects fixed in major open-source projects over the past decade**, through OSS-Fuzz and equivalent infrastructure.

So the recommendation is a single pipeline: a sanitiser-instrumented build, a coverage-guided fuzzer, a persistent corpus, and every crash minimised into a regression test (Module 08 §4) — four components, all free, and the combination is the state of the practice.

- **Run the existing test suite under AddressSanitizer** — which frequently finds defects on the first day, in code that has passed those same tests for years without complaint.
- **Because a test asserts the *output* and a sanitiser asserts the *execution*** — so a test can pass while the program corrupts memory, provided the corruption did not happen to change the answer on that input.
- **Add UBSan to the same build**, since the runtime costs are compatible and the detection overlap is small — so the marginal cost of the second sanitiser is close to nothing.
- **Run a separate ThreadSanitizer build** over the concurrency tests specifically (Module 05 §1), since it cannot share a build with ASan and its cost is high enough to want targeting.
- **And treat any sanitiser report as a build failure rather than a warning** — otherwise the reports accumulate and the output becomes exactly the ignored dashboard of Module 07 §4. **'A test asserts the output; a sanitiser asserts the execution' is the sentence that persuades people**, because it explains concisely why a green test suite proves considerably less than it appears to.

4 Limits, and production

A SANITISER ONLY REPORTS WHAT THE EXECUTION ACTUALLY DID

It is a dynamic technique, so it finds defects on the paths you actually executed and says precisely nothing about the paths you did not — which is exactly why **coverage** (Module 08 §3) is the thing to measure and to report.

And it is emphatically not a verifier. 'The test suite passes under AddressSanitizer' means 'these particular inputs caused no detected violation' — which is a statement about your inputs rather than about your program, and Module 13 is about stating it that way.

Which is the opposite failure mode to static analysis (Module 07 §1): **a sanitiser has essentially no false positives — a report is a real defect — and a very large number of false negatives. Static analysis is the reverse.**

So the three techniques are genuinely complementary rather than redundant: static analysis reasons about all paths imprecisely, sanitisers observe some paths precisely, and code review addresses the design and authorisation defects that neither can see (Module 11) — which is why Project 2 requires all three and Module 13 asks what the three together still miss.

- **The heavy sanitisers are too expensive for production** at their stated costs, **but hardware-assisted ASan and pointer-tagging on ARM are cheap enough to deploy on a fraction of traffic** — which several large mobile platforms now do, and which finds defects that no test reached.
- **A hardened allocator is the cheaper and more portable option** — randomised placement, guard pages, freed-memory quarantine, and metadata separation, at single-digit percentage overhead (Module 02 §3's allocator argument).
- **UBSan's trap mode for whichever checks you can afford**, which converts undefined behaviour into a clean abort rather than into exploitable behaviour (§2).
- **And crash reporting that you actually read**, because **a repeated crash in a parser is a defect report from a stranger** — and a cluster of crashes at one parser offset is frequently somebody probing (CSCE 701 Module 09 §2).
- **Which is the detection-over-prevention argument** (CSCE 701 Module 01 §1's last row) **applied to memory safety**: you will ship defects, so arrange to hear about them. **Reading your crash reports is free detection**, and it is routinely ignored.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
LLVM sanitizer documentation (free)	§1 and §2's reference — what each detects, what it costs, and which combinations are incompatible.

Resource	What it covers for this module
Serebryany et al. — AddressSanitizer (free)	§1's mechanism in the original — the shadow memory design, which explains the limits.
OSS-Fuzz architecture documentation (free)	§3's pipeline as deployed — the sanitiser and fuzzer combination at scale.
Android's HWASan and memory tagging writeups (free)	§4's production detection — the cost figures and the deployment strategy.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Enable AddressSanitizer on an existing project** and run its test suite.
- 2 **Report what it found** in code that was passing its tests.
- 3 **Write an overflow that ASan catches** and one that lands inside another object and is missed.
- 4 **Explain the difference** in terms of the shadow map.
- 5 **Enable UBSan** and find a signed overflow in real code.
- 6 **Try the trap mode** and confirm the abort is deterministic.
- 7 **Try to enable ASan and TSan together** and report what happens.
- 8 **Fuzz a parser with and without sanitisers**, and compare the defect counts.
- 9 **Make a sanitiser report fail your build**, and verify it does.
- 10 **Measure the overhead** of each sanitiser on your own workload.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 How does AddressSanitizer work, and what follows about its limits?
- 2 Name five sanitisers, what each detects, and the rough cost.
- 3 Which two are incompatible, and which can ship in production?
- 4 Name five things UBSan catches, and what trap mode achieves.
- 5 Why does fuzzing without sanitisers miss defects?
- 6 Why does a sanitiser without fuzzing miss defects?
- 7 What does a test assert, and what does a sanitiser assert?
- 8 What does 'the suite passes under ASan' actually mean?

- 9 Contrast the false positive and negative profiles with static analysis.
- 10 Give four ways to run detection in production.